ENT-164 · Intro to Making

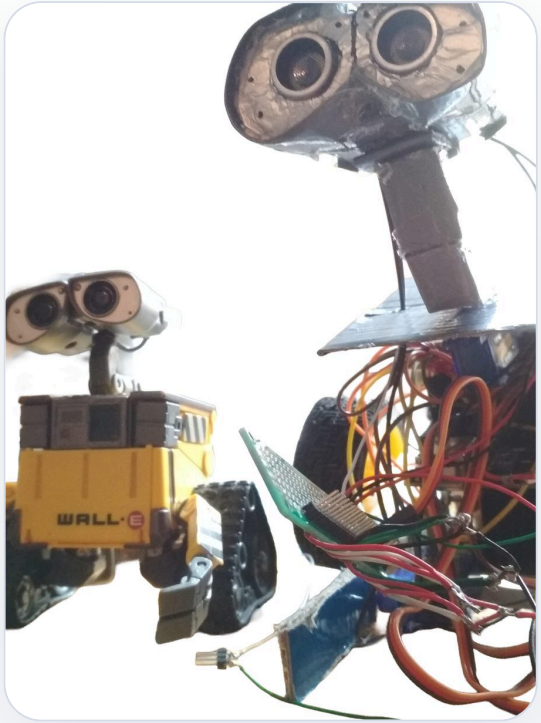
Tufts University | Derby Entrepreneurship Center at Tufts

CLASS 6 · WEEK 6 · FALL 2026

Going Wireless

Your ESP32 can serve a web page your phone opens. No app required. Today we get your robot on WiFi, build it up step by step, and drive it from your phone's browser.

Thu, Oct 15 | JCC 301 / Nolop | No programming required | Bring your ESP32



Thomas Van de Velde · Instructor · **Alexander Beattie** · TA · **Patrice Pinardo** · TA

Tufts University · Gordon Institute

Before we go wireless

You should have all three from last class. Tell us now if something is missing.

01

Electronics kit

You've received your electronics kit. The parts you'll wire up today.

02

opencode + MicroPython

opencode is set up and MicroPython is deployed to your ESP32. It runs your code on the board.

03

AI-assisted electronics

You can already build basic electronics projects with the help of AI.

From your phone to the internet

Connect, reach out, and build. The three things we do in class today.

01

Connect to your microcontroller

Reach your device from a phone and from a web browser. No native app needed.

02

Connect to external services

Give your device internet access, higher data rates, and cloud services.

03

Drive your robot

Put it all together: a control page on your phone that drives the robot you built.

Two ways to connect

Both work. Only one of them skips app development entirely.

`Bluetooth`

Short-range, device to device

Low-power, short-range communication. Like connecting to a phone app. Requires a native mobile app for user interaction.

This is hard. You'd have to build and ship an app.

`WiFi`

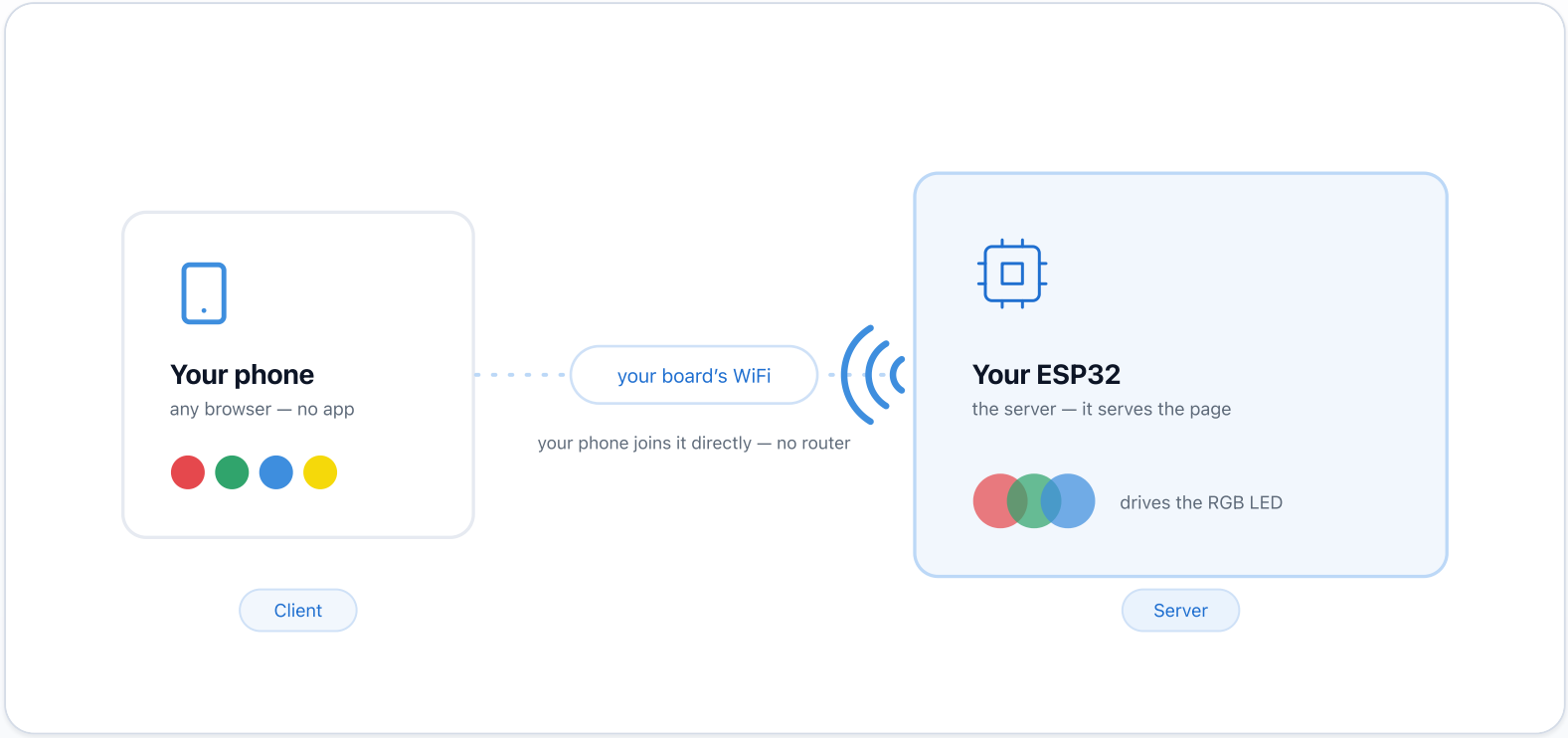
Internet-connected, browser-first

Internet access, higher data rates, and integration with cloud services. Users interact through a simple web browser interface. No app development needed.

Today's approach. The browser is your user interface.

Client and server

Your phone asks for the page; the board serves it — over the board's own WiFi. No router, no cloud.



Client — your phone asks · **Server** — the board answers · **No cloud** — the board is the whole network.

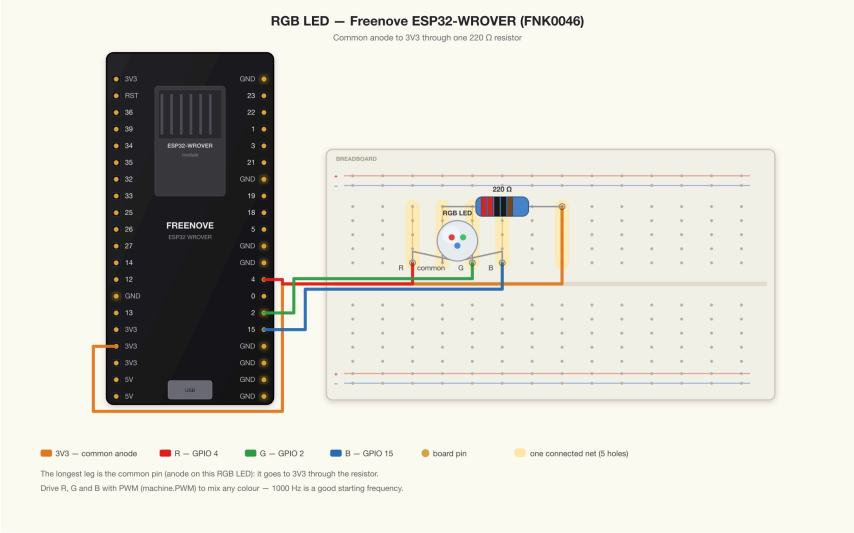
Change a color from your phone

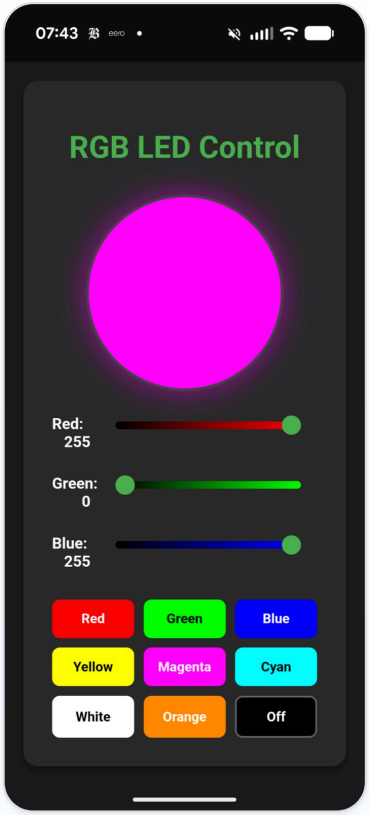
Before the robot, prove the whole loop on one LED: wire it, ask for a page, open it on your phone.

THE PROMPT

“I’ve connected an ESP32. I want to use MicroPython to control an RGB LED from my phone over a wireless hotspot.”

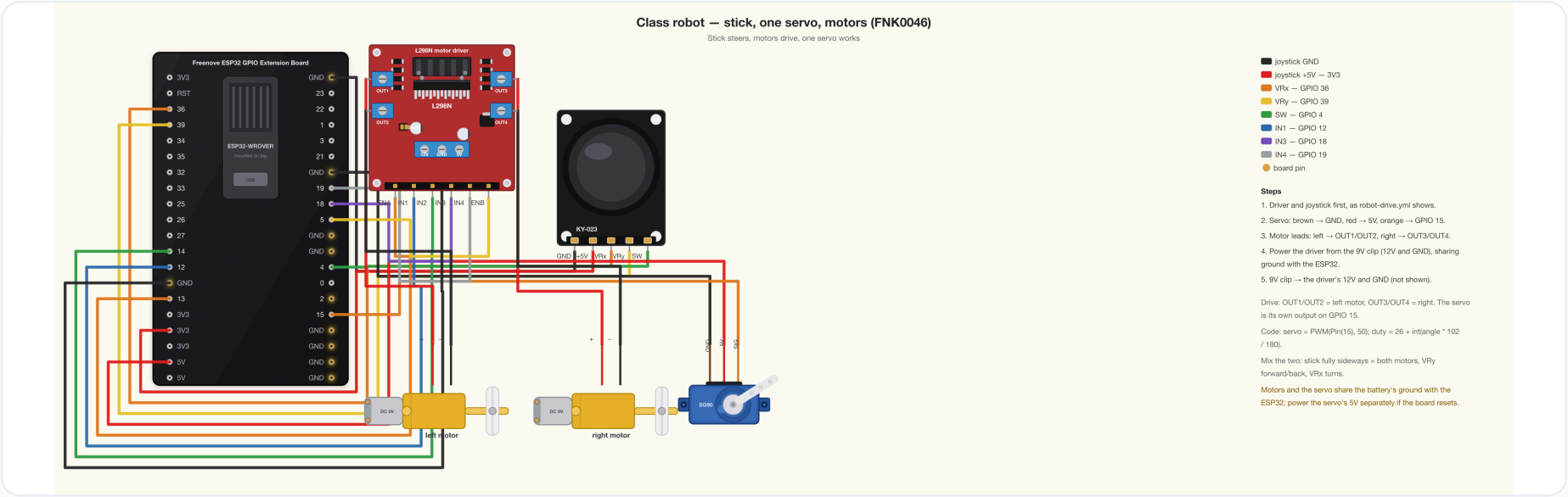
Join the board’s WiFi, open the address it prints, and pick a color.





At Nolop: Wiring the drive train

The L298N sits between your ESP32 and the motors — the same wiring your robot already has.



Wheels **off the ground** while you test. The motors spin the moment the code runs.

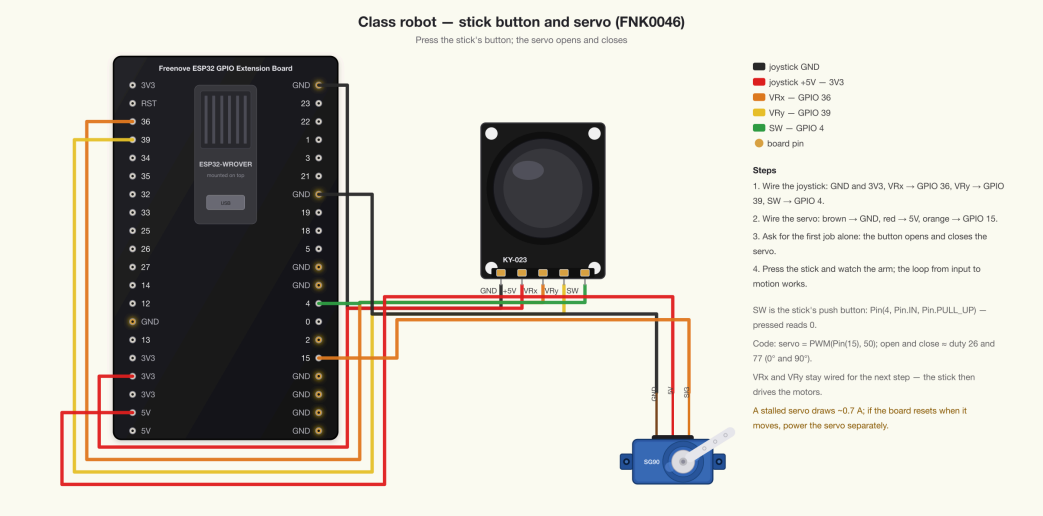
Start with one button and one servo

The smallest piece of the robot: press, and the arm moves. Prove the loop before adding motors.

THE PROMPT

“I want to use my joystick’s push button to open and close the servo.”

One small ask, one test. This is the loop for every step after it.



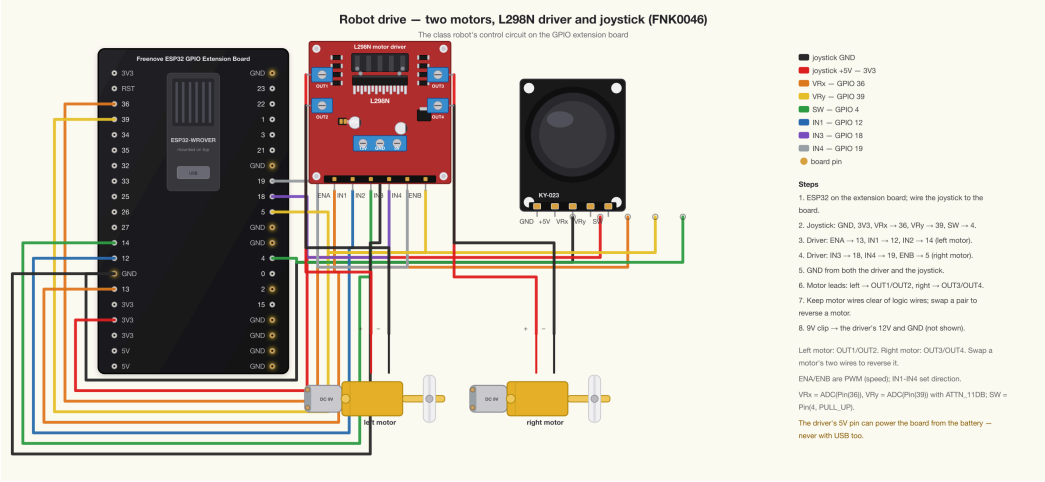
The stick drives

Now the drive train: forward and back, and pivoting turns with differential steering.

THE PROMPT

“Now drive the robot from the joystick: up and down go forward and back, left and right pivot it with differential steering. Stop when the stick is centered. I have a L298N driver to control the motors.”

This is the drive half of the robot — everything from step 1 stays wired.



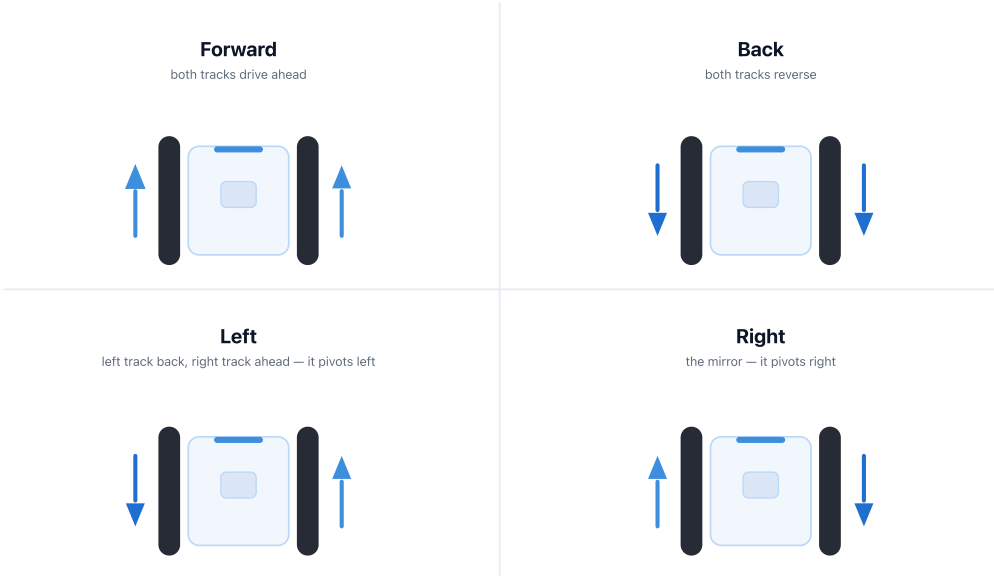
Check it before you trust it

Wheels off the ground first, then every direction on the floor.

THE PROMPT

```
“Before we add WiFi, verify the robot moves
correctly: add a self-test that drives
forward, then backwards, then turns left and
right — about a second each — and prints each
step, so I can watch and check.”
```

Something off? Say exactly what you saw — “the left wheel spins backwards when I push up” — then re-test. One fix at a time.



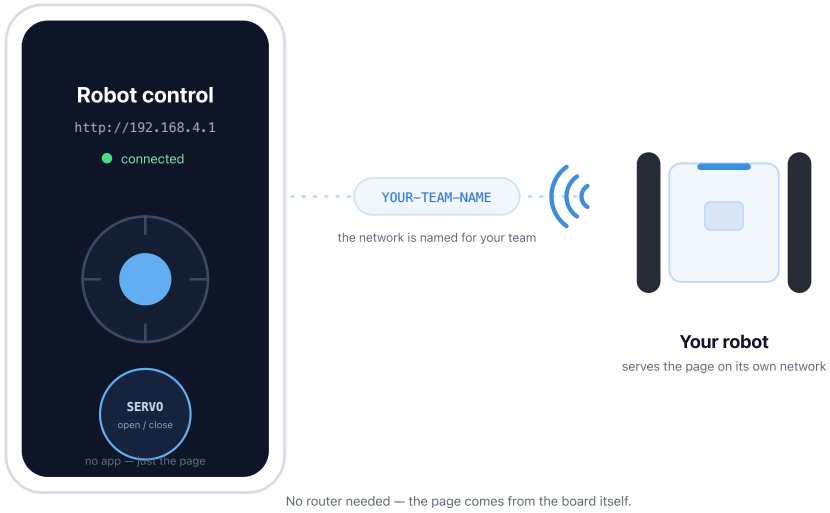
Make the robot the network

The hotspot is your team's name — your phone talks straight to the board.

THE PROMPT

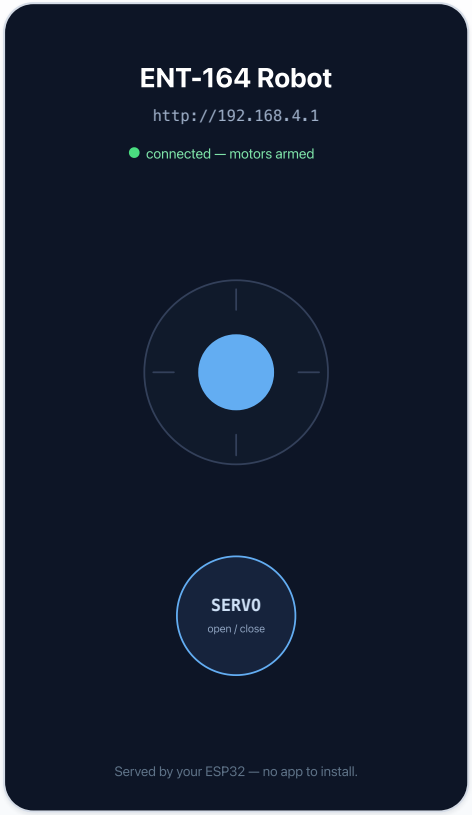
"Finally, make the robot its own network: create a WiFi hotspot on the device named "[fill in your team name]", and serve the joystick and button UI over it. When my phone joins the robot's WiFi, the control page opens and drives the robot — no other network needed."

Replace the highlighted placeholder with your team's name — it becomes the WiFi network everyone joins.



Every move is a request

The page has two controls: the stick drives, the button works the servo.



WHAT EACH CONTROL DOES

Drag the stick → the motors follow your thumb

Let go → everything stops

Press the button → the servo opens or closes

Every drag and press is a request the board answers itself. The whole app is one page — nothing to install.

What happened?

Every tap on your phone becomes a request the ESP32 answers.

A web server starts

1

Your ESP32 runs a web server that renders a web page. No separate computer needed.

The page gets an address

2

The board serves the page from the network it creates — the address the code prints, like `http://192.168.4.1`.

Your phone renders it

3

Your phone's browser renders the HTML that the ESP32's web server generates.

Requests round-trip

4

Interacting with the page sends a request to the ESP32 (its address is `http://192.168.4.1`). It reads the request, sets the pins, and answers with the next page state.

Other things to try

Once your device is online, the browser becomes a remote control for anything you build.

01

Visualize sensor data

A dashboard with temperature history charts.

02

Joystick driving

Swap the buttons for a virtual joystick: drag to steer, release to stop.

03

Manage settings on the phone

Save them to the device: set a thermostat temperature, or set an alarm clock.

04

Device state visualization

See the status of an LED (on/off) or the current servo angle.

BEFORE YOU GO

Questions?

Then get your device online. Next week: Final Robot Assembly and Testing. Integrate everything you've learned into a working robot.

Individual assignment: electronics project. Present a working circuit.

WHERE TO FIND US

Office hours
`go.tufts.edu/meet_tvdv`

Email
`thomas.van_de_velde@tufts.edu`

Course materials
tuftsmaker.github.io/ENT-164